

Herança, Polimorfismo e Encapsulamento

Miguel Costa
miguelangcosta@gmail.com

Herança

Herança permite que **uma classe herde propriedades e métodos de outra classe.**

- **extends** → herança

```
class Funcionario {  
    protected nome: string;  
    protected salario: number;  
  
    constructor(nome: string, salario: number) {  
        this.nome = nome;  
        this.salario = salario;  
    }  
  
    public calcularSalario(): number {  
        return this.salario;  
    }  
}
```

```
class Gerente extends Funcionario {  
    private bonus: number;  
  
    constructor(nome: string, salario: number, bonus: number) {  
        super(nome, salario); // chama o construtor da classe pai  
        this.bonus = bonus;  
    }  
  
    public calcularSalario(): number {  
        return this.salario + this.bonus;  
    }  
}
```

Construtor em classes filhas (**super**)

Quando uma classe herda outra, o construtor da classe filha **precisa chamar o construtor da classe pai** usando **super()**.

```
class Animal {  
    constructor(public nome: string) {}  
}  
  
class Cachorro extends Animal {  
    constructor(nome: string, public raca: string) {  
        super(nome); // chama o construtor da classe pai  
    }  
}
```

Construtor em classes filhas (**super**)

Se a classe filha não tem construtor, então não precisa.

```
class Animal {  
    constructor(public nome: string) {}  
}  
  
class Cachorro extends Animal {  
    // sem construtor  
}  
  
const dog = new Cachorro("Rex");
```

Sobrescrita de métodos (override)

Uma classe filha pode **sobrescrever** um método da classe pai com um comportamento diferente.

```
class Animal {  
    emitirSom() {  
        console.log("Som genérico");  
    }  
}  
  
class Gato extends Animal {  
    emitirSom() {  
        console.log("Miau");  
    }  
}
```

Uso de **readonly** para imutabilidade

Evita que propriedades sejam alteradas depois de definidas.

```
class Pessoa {  
    constructor(public readonly nome: string) {}  
}  
  
const p = new Pessoa("João");  
p.nome = "Maria"; // ✗ erro
```

Classes abstratas (**abstract**)

- **O que é?**
 - Classes que **não podem ser instanciadas**, apenas herdadas.
- **Quando usar?**
 - Quando queremos definir um “modelo” que outras classes devem seguir.

```
abstract class Forma {  
    abstract calcularArea(): number;  
}  
  
class Circulo extends Forma {  
    constructor(public raio: number) {  
        super();  
    }  
  
    calcularArea(): number {  
        return Math.PI * this.raio ** 2;  
    }  
}
```

Classes abstratas vs interfaces

Use interface quando:

- Para **definir o formato de um objeto**
- Para garantir que classes implementem métodos, mas **sem fornecer implementação**
- Para que diferentes classes possam “ser do mesmo tipo” sem estarem numa hierarquia de herança

Vantagens da interface

- Pode ser implementada por **qualquer classe**
- Pode ser usada para tipos (não apenas classes)
- Permite **múltiplas implementações** (uma classe pode implementar várias interfaces)

```
class Bug implements ITarefa { /* ... */ }  
class Feature implements ITarefa { /* ... */ }
```


Classes abstratas vs interfaces

Use classe abstrata quando:

- Para **fornecer código comum** para várias classes
- Para **obrigar** que subclasses implementem métodos específicos
- Para usar **herança**, compartilhando propriedades e métodos

```
abstract class Tarefa {  
    constructor(  
        public titulo: string,  
        public descricao: string  
    ) {}  
  
    abstract calcularEsforco(): number;  
  
    imprimirResumo(): void {  
        console.log(`${this.titulo} - ${this.descricao}`);  
    }  
}
```

Classes abstratas & interfaces

```
interface IValidavel {  
    validar(): boolean;  
}  
  
abstract class Usuario implements IValidavel {  
    constructor(public nome: string) {}  
  
    abstract validar(): boolean;  
}
```

Interfaces e herança (**extends** com interfaces)

```
interface SerVivo {  
    respirar(): void;  
}  
  
interface Mamifero extends SerVivo {  
    amamentar(): void;  
}
```

Composição vs Herança

Muitas vezes **não é recomendado usar herança**, e sim **composição**.

```
class Motor {  
    ligar() { console.log("Motor ligado"); }  
}  
  
class Carro {  
    constructor(private motor: Motor) {}  
  
    ligarCarro() {  
        this.motor.ligar();  
    }  
}
```

Encapsulamento

Encapsulamento é o conceito de **proteger os dados internos de uma classe**, permitindo que eles sejam acessados ou modificados apenas de forma controlada.

Em TypeScript, usamos **modificadores de acesso**:

- **public** → acessível em qualquer lugar (padrão)
- **private** → acessível apenas dentro da classe
- **protected** → acessível dentro da classe e subclasses

Encapsulamento - public

Tudo que é **public**:

- Pode ser acessado fora da classe
- Pode ser lido e alterado livremente

Quando usar?

- Para **comportamentos que fazem parte da classe**
- Métodos que outras partes do sistema precisam chamar

Encapsulamento - public

Tudo que é **public**:

- Pode ser acessado fora da classe
- Pode ser lido e alterado livremente

Quando usar?

- Para **comportamentos que fazem parte da classe**
- Métodos que outras partes do sistema precisam chamar

```
class Pessoa {  
  public nome: string;  
  
  constructor(nome: string) {  
    this.nome = nome;  
  }  
  
  public falar(): void {  
    console.log(`Olá, meu nome é ${this.nome}`);  
  }  
}  
  
const pessoa = new Pessoa("Maria");  
console.log(pessoa.nome); // OK  
pessoa.falar();           // OK
```

Encapsulamento - **private**

Tudo que é **private**:

- Só pode ser acessado **dentro da própria classe**
- Nem mesmo classes filhas conseguem acessar

```
class Usuario {  
    private senha: string;  
  
    constructor(senha: string) {  
        this.senha = senha;  
    }  
}  
  
const user = new Usuario("123456");  
user.senha; // ✗ ERRO: propriedade privada
```


Encapsulamento - **protected**

Tudo que é **protected**:

Pode ser acessado:

- Dentro da classe
- Dentro das classes filhas

✗ Não pode ser acessado:

- Fora da hierarquia de herança

```
class Funcionario {  
    protected salario: number;  
  
    constructor(salario: number) {  
        this.salario = salario;  
    }  
}  
  
class Gerente extends Funcionario {  
    calcularBonus(): number {  
        return this.salario * 0.2; // ✓ OK  
    }  
}  
  
const gerente = new Gerente(3000);  
gerente.salario; // ✗ ERRO
```

Encapsulamento

Erros Comuns:

- Tudo como **public**
 - Falta de segurança e controle
- Tudo como **private**
 - Herança fica inútil

Boa prática

- **private** → dados internos
- **protected** → dados que subclasses precisam
- **public** → métodos de uso externo

Polimorfismo

Polimorfismo significa "**muitas formas**".

Em POO, significa que **classes diferentes podem responder de forma diferente ao mesmo método**.

```
const funcionario = new Funcionario("Ana", 2000);  
const gerente = new Gerente("Carlos", 3000, 1000);  
  
console.log(funcionario.calcularSalario()); // 2000  
console.log(gerente.calcularSalario());    // 4000
```

Polimorfismo com arrays

O TypeScript chama o método correto automaticamente.

```
const funcionarios: Funcionario[] = [  
  new Funcionario("Ana", 2000),  
  new Gerente("Carlos", 3000, 1000)  
];  
  
funcionarios.forEach(f => {  
  console.log(f.calcularSalario());  
});
```

Polimorfismo

Pode ser:

- **Por override (herança)**
Ex.: **Cachorro** sobreescreve **emitirSom()**
- **Por interfaces**
Ex.: objetos diferentes implementam a mesma interface

```
interface Pagavel {  
    pagar(): void;  
}  
  
class Fatura implements Pagavel {  
    pagar() { console.log("Fatura paga"); }  
}  
  
class Salario implements Pagavel {  
    pagar() { console.log("Salário pago"); }  
}
```

Questões